

# Extracting Correspondences and Geometry from Pretrained Scene Representation Transformers

Alex Hu  
MIT

32 Vassar Street Cambridge, MA 02139  
alexhu@mit.edu

Kairo Morton  
MIT

32 Vassar Street Cambridge, MA 02139  
mortonk@mit.edu

## Abstract

*Given posed multi-view images of a static scene, we aim to infer view correspondences and 3D geometry from the inference time activations of a pre-trained Scene Representation Transformer ([1]). To this end, we are able to reconstruct colored point clouds which aggregate both the appearance and geometry within the 3D scene.*

## 1. Introduction

Among the variety of neural scene representations present in the 3D computer vision literature, parameterizing scenes as a light field has been shown to have a multitude of efficiency benefits over more common volumetric representations. Recently, the Scene Representation Transformer (SRT) and related works [2] have used an encoder-decoder transformer architecture to predict the color seen by any ray in a scene given only a few image observations as input. Unfortunately, parameterizing a 3D scene using a light field places the sole focus on appearance modelling without a clear method for extracting geometric information about the underlying 3D scene. Furthermore, for many downstream tasks such as navigation and planning this geometric information is critical. We believe our method with additional improvements can be applied to these domains.

As such, in this work we aim to leverage the multi-view consistent rendering and light field parameterization of the Scene Representation Transformer to first extract multi-view correspondences and subsequently use these correspondences for scene geometry reconstruction. The SRT Our key contributions are as follows:

- A method for extracting multi-view correspondences from the Scene Representation Transformer
- An implementation of triangulation to extract scene geometry from the Scene Representation Transformer in the form a colored point cloud

## 2. Method

### 2.1. Extracting Ray-Patch Correspondences

Light Field Transformers such as the SRT represent a given 3D scene as set of tokens known as set-latents. These set-latent tokens are produced using a Transformer encoder which processes the five posed input views similarly to a Vision Transformer, thus, each set-latent token corresponds to an 8x8 patch of pixels in one of the 5 input views. However, unlike Vision Transformers, a small CNN precedes the Transformer encoder to embed patches of pixels into tokens while incorporating the positional information of the camera rays passing through said patch (see original paper for more details). Given the positional information stored in each set-latent token predicting the color of a query ray is done in the SRT via two layers of cross-attention between the query ray and the set latents. During this rendering process we show that the query ray tokens typically attend most strongly to the set-latent tokens which correspond to input image patches containing the projection of the 3D surface point that the query ray intersects first. Thus, it is clear that SRT is able to implicitly learn complex geometric relationships using an unstructured internal representation and a novel view synthesis training objective

It is exactly this insight which enables our method to extract multi-view correspondences. Specifically, after encoding the input views in to the set-latent scene representation for any query ray  $r$  we start by extracting the attention vector  $a_{l,i} \in \mathbb{R}^n$  at the  $l^{\text{th}}$  layer and  $i^{\text{th}}$  attention head in decoder network. In the above definition  $n$  denotes the total number of set-latent tokens produced by the transformer encoder. We can then define the input patch that corresponds to ray  $r$  as follows:

$$p^* = \arg \max a_{l,i} \quad (1)$$

While this works well generally, in some cases we want to additionally determine correspondence quality. In such instances the correspondence quality score  $s = \max a_{l,i}$ .

This score can then be used when computing the ray-patch correspondences for a collection of rays from the same layer and head in the decoder to filter spurious correspondences.

## 2.2. Generating Point Clouds via Triangulation

Using the ray-patch correspondences defined in the previous section we devise a method to reconstruct dense point clouds. In particular, the previous section lays out a well defined method to compute ray-patch correspondences for an arbitrary ray in the scene. Therefore we can simply render an arbitrary number of views in order to find multiple ray-patch correspondences. For each ray-patch correspondence we solve an approximate triangulation problem to find the 3D surface point  $x^*$  represented by the ray-patch correspondence. Unfortunately, a ray-patch correspondence is not specific enough to define an exact triangulation problem as triangulation requires ray-ray correspondence instead of ray-patch. However, since the input views are posed each patch corresponds to 64 well defined rays. As such, for a ray-patch correspondence we solve the triangulation problem for each of the 64 possible ray-ray correspondences and keep the 3D point with the lowest error. More concretely, we solve 64 non-linear least squares optimization problem of the following form for a possible ray-ray correspondence between rays  $r_1$  and  $r_2$ :

$$x^* = \arg \min \text{dist}(r_1, x) + \text{dist}(r_2, x) \quad (2)$$

After repeating this process for each ray-patch correspondences we are left with 3D points defining a dense point cloud representation of our 3D scene. Finally, to combine the appearance and geometry information from the SRT we color the point cloud using the predicted colors for each rendered ray that defined a point via a ray-ray correspondence. Additionally, due to outlier or incorrect correspondences, the point cloud can be filtered using the distance errors at the end of the optimization process. This allows for the removal of outlier points at the expense of reducing the point cloud density.

## 3. Experiments

The following two experiments illustrate the two key contributions of our method: extracting ray-patch correspondences from the pretrained SRT model and using these correspondences to compute colored point clouds. Both of the following experiments are performed using a pretrained version of the SRT using the improved encoder described in A.4 of Sajjadi et al. [3]. In addition, all attention maps extracted for the results shown below are from the 2<sup>nd</sup> attention head in the 1<sup>st</sup> cross-attention layer of the decoder network. Finally, to evaluate our method we chose to show results for various scenes in the Multishapenet dataset.

### 3.1. Visualizing Multi-View Correspondences

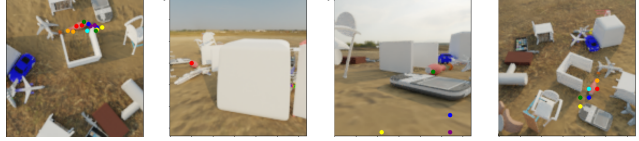


Figure 1. This figure shows ray-patch correspondences between input views. The point in the far left image denote query ray locations and the corresponding point in the subsequent images denote corresponding patch centers

In this experiment we aimed to explore the quality of ray-patch correspondences by visualizing the correspondences between input views. In order to accomplish this we follow the approach described in section 2.1 for extracting ray-patch correspondences, however, we get the decoder attention maps by re-rendering the input views. Thus, for every ray in the input views we compute the corresponding patch in the the 4 other input views by excluding set latents derived from patches in the input view being re-rendered. An example of this a this ray-patch attention can be seen in Figure 1 using only 3 of the 4 other input views. In Figure 1 while most of the correspondences on the pink object are consistent across the multiple input views there exist query rays that intersect the ground plane which do not correspond the correct patch in the other input views. We note this as possible evidence that SRT stores information about the ground in set-latent tokens with less geometric information as differentiating ground points is not necessary to minimize the ray color prediction loss for this dataset which has relatively constant ground texture.

### 3.2. 3D Point Clouds from Correspondences

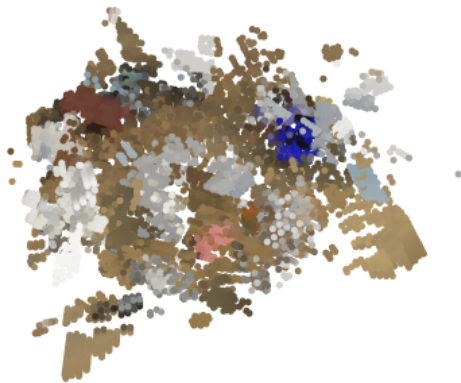


Figure 2. This figure shows one view of the colored point cloud created using the ray-ray correspondences between input views

In addition to evaluating the correspondences extracted from the model visually, we also used them to perform col-

ored point cloud reconstruction via the method described in 2.2. The results from this process are shown in Figure 2. It is clear that the point cloud is noisy but it still manages to capture the high-level geometric structure of the scene.

## 4. Conclusion

All in all, our work shows that the Scene Representation Transformer does in fact learn to encode geometric information implicitly through the process of modelling the appearance of 3D scenes. Additionally, we have shown that this geometric information can be extracted through the model’s attention maps and used for tasks such as point cloud reconstruction. In future work, we hope to explore whether this geometric information is maintained in the unposed variant of the SRT (UpSRT), possibly allowing for insights in camera pose regression and other tasks.

## References

- [1] Mehdi S. M. Sajjadi et al. Scene representation transformer: Geometry-free novel view synthesis through set-latent scene representations. *CVPR*, 2022. 1
- [2] Mehdi S. M. Sajjadi et al. Rust: Really unposed srt, 2023. 1
- [3] Mehdi S. M. Sajjadi, Daniel Duckworth, Aravindh Mahendran, Sjoerd van Steenkiste, Filip Pavetić, Mario Lučić, Leonidas J. Guibas, Klaus Greff, and Thomas Kipf. Object scene representation transformer, 2022. 2